Bags, Vectors & Transformers

Convolutional Neural Networks for Text · Bonus, Part 7

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

The Problem: Bags Forget Order

"tax cut for schools"
"school cut for tax"

Different meaning, identical bag of words.

"not good, actually great"
"not great, actually good"

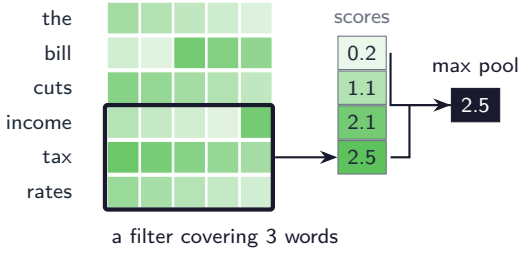
Same words, different emphasis.

Day 1 offered one fix: **n-grams** (word pairs, triples). They help, but:

- the number of features explodes
- each n-gram is learned separately: *income tax* teaches the model nothing about *payroll tax*

A **convolutional neural network** learns which short phrases matter, and works on embeddings, so similar phrases are treated similarly.

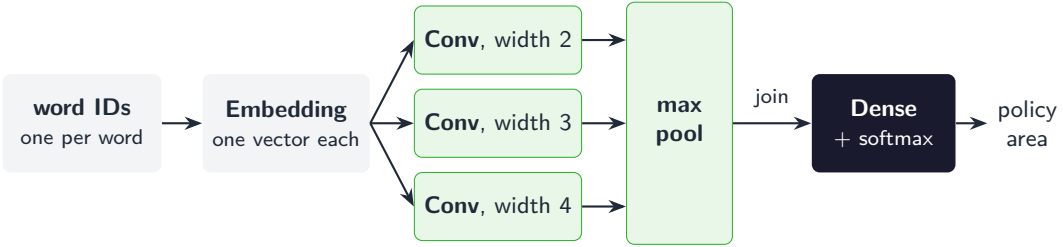
A Sliding Window over Word Vectors



1. Each word is a **vector**, so a sentence is a **matrix**
2. A **filter** covers a few words and slides along, scoring each position: how well does this window match my pattern?
3. **Max pooling** keeps the best score: does the pattern occur *anywhere*?

A layer has many filters (say 128), all starting random. Training turns each one into a detector for phrases that help with the task, such as *income tax* or *health insurance*.

The Full Architecture



Several window sizes side by side (Kim, 2014): bigram, trigram and four-gram detectors look at the same text in parallel.

In Keras this needs the **functional API** (branches); a single window size fits in a `Sequential` model. In PyTorch it's a `ModuleList`.

You Can Read What a Filter Learned

Run one filter over every training title and list the windows where it fires most strongly:

filter 2	*reform excise taxes* | *income tax for* | *internal revenue code*
filter 3	*the armed forces* | *armed forces and* | *military construction for*
filter 4	*mental health services* | *amend health insurance* | *prescription drug costs*
filter 8	*higher education act* | *student loans and* | *public schools for*

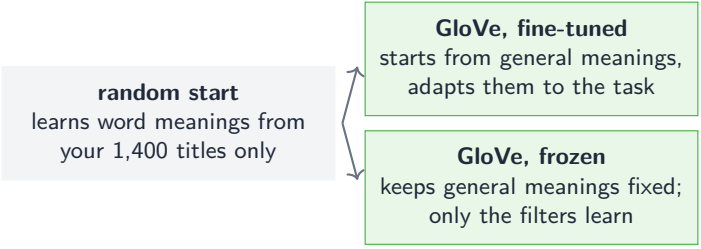
Illustrative output; your filters will differ from run to run.

A useful sanity check: if the top phrases were bill numbers or dates, the model would be learning a shortcut.

But not an explanation: many filters respond to mixed phrases, and the output layer weighs them all. Treat filters like topics in a topic model: exploration, not measurement.

A Head Start: Pre-trained Embeddings

The embedding layer can start from random numbers, or from **GloVe** vectors (Day 2), learned from billions of words. That's transfer learning in its simplest form.



With small training sets, pre-trained vectors usually help: the model already knows that *Medicare* and *Medicaid* are related. BERT (Day 3) takes this idea all the way.

Where CNNs Stand Today

From about 2014 to 2018, CNNs were among the best text classifiers. Then pre-trained transformers overtook them on almost every benchmark.

Still useful because they are...

- **small and fast**: millions of posts on a laptop
- **strong on short texts** with clear keywords
- **easier to inspect** than attention heads

Limited because they...

- only see **local** context (a few words)
- start with little knowledge of language
- lose to fine-tuned transformers on subtle tasks

And convolution is far from dead: it remains the workhorse for images (Bonus, Part 9).

In the Notebook

- A convolution and max pooling, **by hand**, on a six-word sentence
- Turning titles into padded sequences of word IDs, once, for both stacks
- A text CNN in **Keras** (Sequential and functional API) and in **PyTorch**
- Starting from **GloVe**: fine-tuned vs. frozen
- Reading off the phrases each filter detects
- Comparing with the TF-IDF baseline and Day 3, on the same test set

`bonus/notebooks/7_cnns_for_text_exercises.ipynb` · 6 exercises · CPU is fine